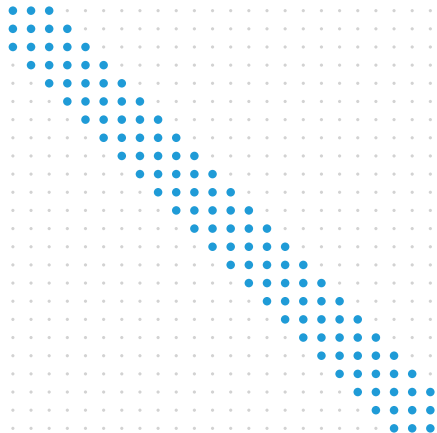


Métodos Iterativos para Sistemas de Ecuaciones Lineales

Eficiencia algorítmica y convergencia en
modelos de gran escala

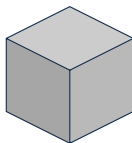
Autor: Dr. Hermes Pantoja Carhuavilca

Curso: Métodos Numéricos



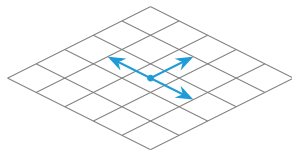
La Necesidad de la Iteración: El Problema de la Escala

Métodos Directos (Elim. Gaussiana)



- Exactos en teoría (pasos finitos).
- Costo computacional: $O(N^3)$.
- Ineficientes para matrices dispersas (almacenan ceros).
- **Veredicto:** ideales para $N < 100$.

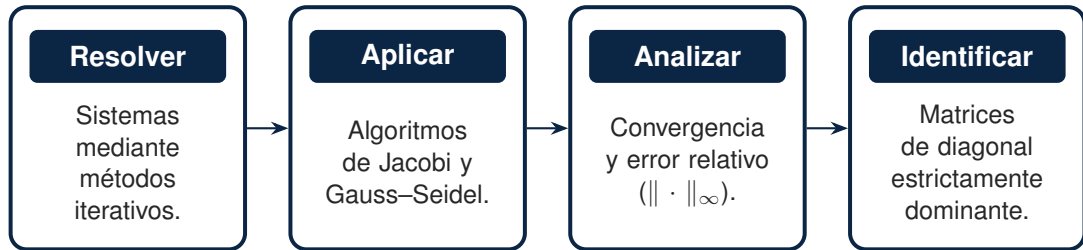
Métodos Iterativos (Jacobi, Gauss–Seidel)



- Aproximados (sucesión $\mathbf{x}^{(k)} \rightarrow \mathbf{x}$).
- Costo reducido: solo operan con elementos no nulos.
- Uso eficiente de memoria (vital en fluidos/calor).
- **Veredicto:** estándar para $N > 1000$.

Insight: para grandes sistemas, los métodos iterativos evitan el desbordamiento de memoria al ignorar los millones de ceros de la matriz.

Logro de la Sesión

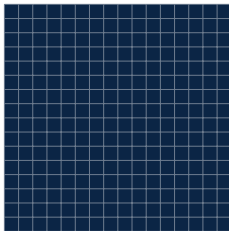


Al finalizar la sesión, el estudiante dominará estas cuatro competencias fundamentales para el análisis numérico.

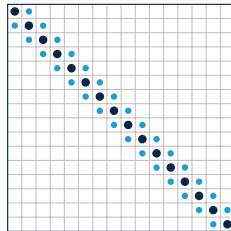
Motivación: ¿Por qué Métodos Iterativos?

- **Escalabilidad:** sistemas reales de ingeniería con miles de variables.
- **Costo computacional:** los métodos directos ($O(n^3)$) consumen demasiada memoria y tiempo.
- **Matrices dispersas:** ideales para estructuras con muchos ceros (elementos finitos, diferencias finitas).

Matriz densa



Matriz dispersa



Eficiencia: solo se almacenan los no nulos.

$$Ax = b$$



A

Matriz de coeficientes
($n \times n$)



x

Vector de incógnitas
($n \times 1$)

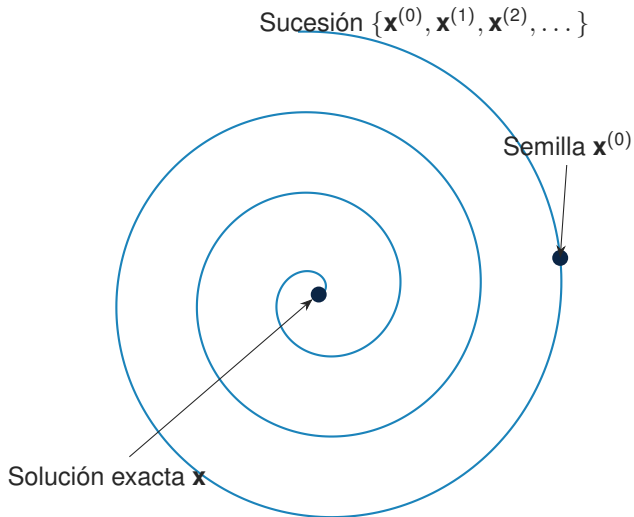


b

Vector de términos
independientes ($n \times 1$)

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

Idea Fundamental de la Iteración



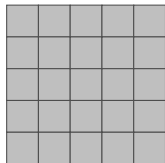
Objetivo: convergencia

$$\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}$$

En lugar de un cálculo único, generamos una **secuencia de refinamiento progresivo**.

Descomposición de la Matriz

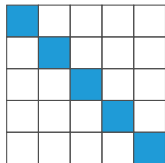
$$A = D - L - U$$



A

Matriz original

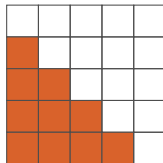
=



D

Diagonal

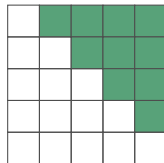
−



L

Triangular inferior

−



U

Triangular superior

D : diagonal de A .
superior de A .

$-L$: parte estrictamente inferior de A .

$-U$: parte estrictamente

Fundamentación Teórica

De $A\mathbf{x} = \mathbf{b}$ a $\mathbf{x} = T\mathbf{x} + \mathbf{c}$

Los métodos iterativos transforman el sistema lineal en un **problema de punto fijo**, generando una secuencia $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots$ que converge a la solución.

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}}_A = \underbrace{\begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix}}_D - \underbrace{\begin{bmatrix} 0 & 0 & \cdots & 0 \\ -a_{21} & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ -a_{n1} & \cdots & -a_{n,n-1} & 0 \end{bmatrix}}_L - \underbrace{\begin{bmatrix} 0 & -a_{12} & \cdots & -a_{1n} \\ 0 & 0 & \ddots & \vdots \\ \vdots & & \ddots & -a_{n-1,n} \\ 0 & 0 & \cdots & 0 \end{bmatrix}}_U$$

Idea

Escribir $A = M - N$ con M fácil de invertir:

$$M\mathbf{x} = N\mathbf{x} + \mathbf{b} \Rightarrow \mathbf{x}^{(k+1)} = M^{-1}N\mathbf{x}^{(k)} + M^{-1}\mathbf{b}.$$

Forma Iterativa General

Nueva aproximación

Aproximación actual

$$\mathbf{x}^{(k+1)} = \mathbf{T} \mathbf{x}^{(k)} + \mathbf{c}$$

Matriz de iteración
(determina si el método converge)

Vector constante
derivado de **b**

Método de Jacobi: Desplazamiento Simultáneo

Actualización paralela

Para calcular las variables de la iteración actual ($k + 1$) se usan **exclusivamente** los valores de la iteración anterior (k).

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

$$x_1^{(k+1)} = \frac{1}{a_{11}} [b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \dots - a_{1n}x_n^{(k)}]$$

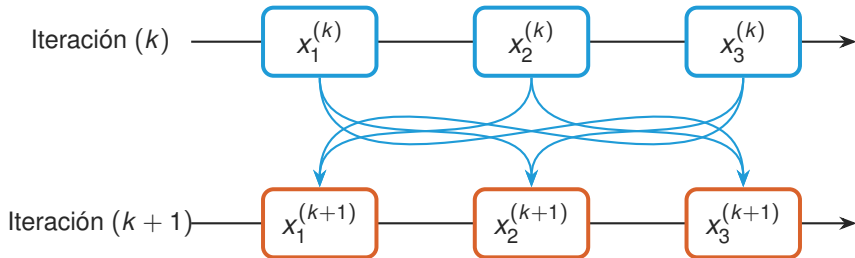
$$x_2^{(k+1)} = \frac{1}{a_{22}} [b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)} - \dots - a_{2n}x_n^{(k)}]$$

$$\vdots$$

$$x_n^{(k+1)} = \frac{1}{a_{nn}} [b_n - a_{n1}x_1^{(k)} - \dots - a_{n,n-1}x_{n-1}^{(k)}]$$

Método de Jacobi: Concepto

Método de desplazamientos simultáneos



Aísla cada incógnita usando **únicamente** valores de la iteración anterior (k).
Todo el vector se actualiza en un solo paso.

Jacobi: Formulación Matricial

Partimos de: $(D - L - U) \mathbf{x} = \mathbf{b}$



Despejamos la diagonal: $D\mathbf{x} = (L + U) \mathbf{x} + \mathbf{b}$



Forma iterativa:

$$\mathbf{x}^{(k+1)} = D^{-1}(L + U) \mathbf{x}^{(k)} + D^{-1}\mathbf{b}$$

Matriz de iteración: $T_J = D^{-1}(L + U)$

Vector constante: $\mathbf{c}_J = D^{-1}\mathbf{b}$

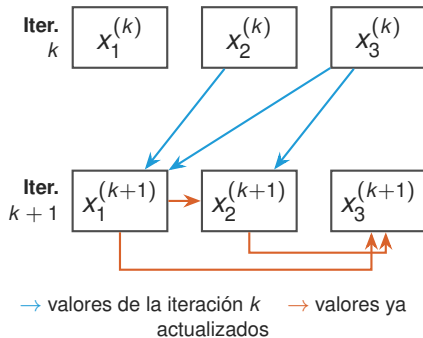
Método de Gauss–Seidel: Desplazamiento Sucesivo

Actualización inmediata

Optimización: tan pronto como se calcula un nuevo valor $x_i^{(k+1)}$, se usa **inmediatamente** en las siguientes ecuaciones de la misma iteración.

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right)$$

Nota: generalmente converge **alrededor de 2 veces** más rápido que Jacobi al usar la información más reciente.



Gauss–Seidel: Formulaci3n Matricial

Agrupamos lo nuevo a la izquierda: $(D - L)\mathbf{x}^{(k+1)} = U\mathbf{x}^{(k)} + \mathbf{b}$

Forma iterativa:

$$\mathbf{x}^{(k+1)} = (D - L)^{-1}U\mathbf{x}^{(k)} + (D - L)^{-1}\mathbf{b}$$

Matriz de iteraci3n: $T_{GS} = (D - L)^{-1}U$

Vector constante: $\mathbf{c}_{GS} = (D - L)^{-1}\mathbf{b}$

Nota: $(D - L)$ es triangular inferior, lo que permite resolver eficientemente mediante **sustituci3n hacia adelante** (no se invierte expl3citamente).

Comparación: Jacobi vs. Gauss–Seidel

Característica	Jacobi	Gauss–Seidel
Convergencia	Generalmente lenta	Usualmente más rápida
Memoria	Requiere vector auxiliar (valores viejos)	Sobrescribe valores (un solo vector)
Paralelismo	Alto (cálculos independientes)	Bajo (secuencial)
Estabilidad	Menor	Mayor

Ejemplo: Jacobi vs. Gauss–Seidel

$$-7x_1 + 2x_2 + 3x_3 = 6$$

$$5x_1 + 9x_2 + x_3 = 26$$

$$x_1 + 4x_2 - 6x_3 = -9$$

$$\mathbf{x}^{(0)} = (0, 0, 0)^T$$

$$\text{Tolerancia: } E^{(k)} < 10^{-6}$$

Solución exacta:

$$\mathbf{x} = (1, 2, 3)^T$$

Jacobi: 29 iteraciones
Gauss–Seidel: 14 iteraciones

Jacobi

k	x_1	x_2	x_3	$E^{(k)}$
1	-0.857143	2.888889	1.500000	1.000000
2	0.611111	3.198413	3.283069	0.543110
3	1.463719	2.184597	3.734127	0.271500
4	1.367368	1.660809	3.200351	0.166787
5	0.988953	1.773645	2.835101	0.133475

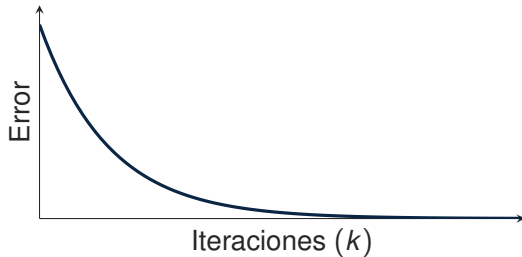
Gauss–Seidel

k	x_1	x_2	x_3	$E^{(k)}$
1	-0.857143	3.365079	3.600529	1.000000
2	1.647392	1.573612	2.823640	0.886988
3	0.802592	2.129267	3.053276	0.276686
4	1.059766	1.960877	2.983879	0.086188
5	0.981913	2.011840	3.004879	0.025909

Análisis de Convergencia

Un método iterativo converge si la diferencia entre la solución real y la aproximación tiende a cero:

$$\lim_{k \rightarrow \infty} \|\mathbf{x}^{(k)} - \mathbf{x}\| = 0$$



¿Qué propiedades de la matriz garantizan esto?

Condiciones de Éxito: Dominancia Diagonal

Condición suficiente: el método converge si el valor absoluto del elemento diagonal es mayor que la suma de los valores absolutos del resto de la fila.

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad i = 1, \dots, n$$

$$A = \begin{bmatrix} -7 & 2 & 3 \\ 5 & 9 & 1 \\ 1 & 4 & -6 \end{bmatrix}$$

$$\text{Fila 1: } |-7| > 2 + 3 \rightarrow 7 > 5 \quad (\text{cumple})$$

$$\text{Fila 2: } |9| > 5 + 1 \rightarrow 9 > 6 \quad (\text{cumple})$$

$$\text{Fila 3: } |-6| > 1 + 4 \rightarrow 6 > 5 \quad (\text{cumple})$$

La matriz es estrictamente diagonal dominante $\Rightarrow$ convergencia garantizada (Jacobi y Gauss–Seidel, para cualquier $x^{(0)}$).

Criterio Espectral de Convergencia

El método converge para cualquier $\mathbf{x}^{(0)}$ **si y solo si:**

$$\rho(T) < 1$$

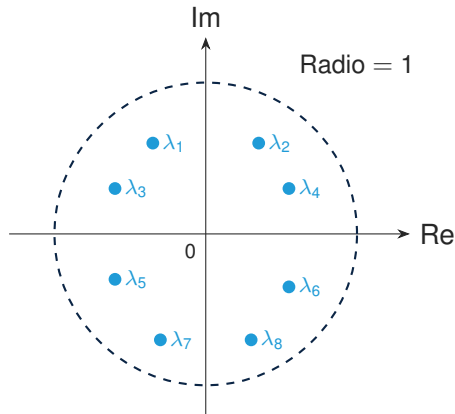
$\rho(T)$ es el **radio espectral**:

$$\rho(T) = \max_i |\lambda_i(T)|$$

Para el ejemplo anterior:

$$\rho(T_J) \approx 0.603, \quad \rho(T_{GS}) \approx 0.303$$

Menor $\rho \Rightarrow$ convergencia más rápida.



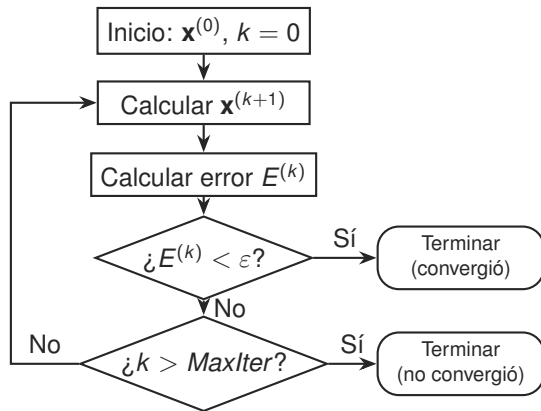
Norma infinito (la componente de mayor magnitud):

$$\|\mathbf{x}\|_{\infty} = \max_{1 \leq i \leq n} |x_i|$$

$$E^{(k)} = \frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_{\infty}}{\|\mathbf{x}^{(k+1)}\|_{\infty}}$$

Mide el **cambio relativo máximo** entre iteraciones sucesivas.

Criterios de Parada

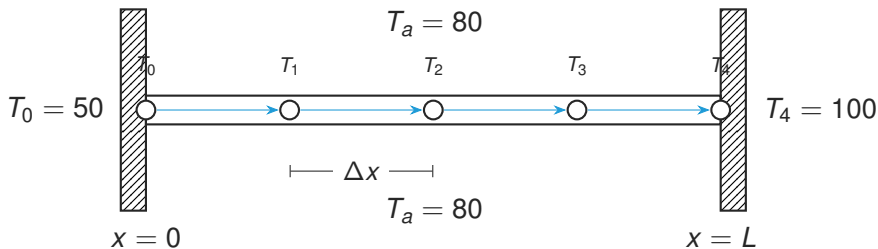


```
k = 0; E = Inf;
while E > tol
    % actualizar x
    k = k + 1;
    if k > maxIter
        break
    end
end
```

Dos condiciones: **precisión** alcanzada o **límite** de iteraciones.

Aplicación en Ingeniería: Transferencia de Calor

Modelado de estado estable (ecuación de Poisson/Laplace)



1. **Discretización:** se divide el dominio físico en una malla de nodos.
2. **Interacción local:** cada nodo depende solo de sus vecinos inmediatos:

$$-T_{i-1} + (2 + h'\Delta x^2) T_i - T_{i+1} = h'\Delta x^2 T_a$$

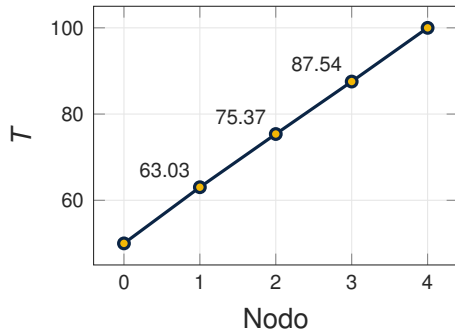
3. **Resultado:** una matriz dispersa (bandeada) donde la mayoría de elementos son cero.

Sistema Resultante y Solución por Gauss–Seidel

Con $h'\Delta x^2 = 0.04$ se obtiene un sistema **tridiagonal**:

$$\begin{bmatrix} 2.04 & -1 & 0 \\ -1 & 2.04 & -1 \\ 0 & -1 & 2.04 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix} = \begin{bmatrix} 53.2 \\ 3.2 \\ 103.2 \end{bmatrix}$$

- Diagonal dominante ($2.04 \geq 1 + 1$, estricta en filas 1 y 3).
- Gauss–Seidel con $\varepsilon = 10^{-4}$: **14 iteraciones.**



Actividad Práctica: El Copiloto Digital

Instrucciones para generación de código (MATLAB):

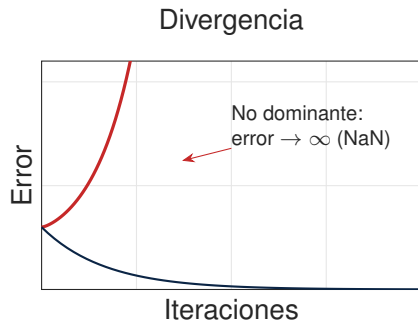
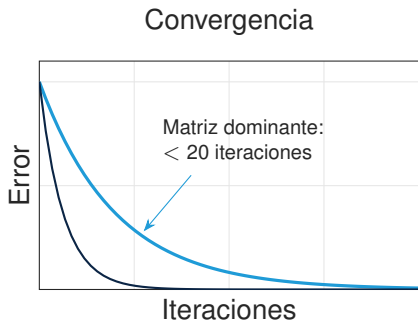
Prompt para copiloto



1. Genera un script en MATLAB para resolver un sistema lineal $Ax=b$ de 100×100 .
2. Implementa el método de Gauss-Seidel.
3. Restricción crítica: asegura que la matriz aleatoria A sea diagonalmente dominante para garantizar convergencia.
4. Define criterio de parada: $\text{tolerancia} < 1e-6$.

Validación Crítica y Análisis de Estabilidad

La IA escribe el código, el ingeniero valida la física



- ☐ ¿Incluye el código un límite `max_iter`?
- ☐ ¿Se verifica la tolerancia?
- ☐ ¿Se comprueba la dominancia diagonal (o $\rho(T) < 1$) antes de iterar?

Referencias Bibliográficas

1. Chapra, S. C., & Canale, R. P. *Métodos Numéricos para Ingenieros*. McGraw-Hill Education. (Análisis de error y algoritmos).
2. Burden, R. L., & Faires, J. D. *Análisis Numérico*. Cengage Learning. (Teoría de radio espectral).
3. Nakamura, S. *Métodos Numéricos Aplicados con Software*. Prentice Hall. (Aplicaciones en transferencia de calor).